

AlgoTutorGen: Contract-Guided Compositional Synthesis of Verifiable Interactive Algorithm Tutors

Anonymous submission

Abstract

LLM-generated algorithm tutors can display a correct answer while encoding an incorrect trace, unreachable controls, broken feedback, or teaching actions that mutate algorithm state. We call this coupling of heterogeneous obligations in one unconstrained HTML output *constraint entanglement*. ALGO-TUTORGEN applies contract-guided semantic factorization: an executable specification yields a validated trace and scene, deterministic compilation supplies the runtime, and a separate overlay handles teaching. We characterize two conditional properties, compositional semantic preservation and pedagogical noninterference, and distinguish them from finite implementation evidence. On 200 tasks from 23 algorithm families, ALGO-TUTORGEN achieves 198/200 complete MACHINE OK pages versus 98/200 for Direct HTML, although both display the expected answer on 200/200 pages. Across 294 artifacts, all 55,108 evaluated Trace–SceneGraph–Runtime frames preserve canonical semantic projections. The contracts reject 2,198/2,198 tested semantic violations and accept 392/392 semantics-preserving transformations. A browser stress suite executes 1,561,298 actions over 240 pages without finding a pedagogical noninterference counterexample. LOCAL RESUME nevertheless shows no significant advantage over GLOBAL RESTART because repair redoes materialization and teaching. Thus semantic and verification decoupling are realized, while full computational recovery decoupling remains open. We make no claim about student learning outcomes. Project page: <https://algenlab.github.io/AlgoTutorGen/>.

Introduction

Large language models (LLMs) can produce an algorithm-teaching webpage that looks complete and displays the expected answer. That surface success is a weak proxy for an executable tutor. The page may encode an incorrect intermediate state, fail to load, expose controls that cannot be reached, return the same message for correct and incorrect answers, omit a hint or learning log, or let a pedagogical action corrupt the algorithm state. This distinction is visible in our main evaluation: ALGO-TUTORGEN and a Direct HTML baseline both display the expected answer on 200/200 pages, but only 198/200 and 98/200 pages, respectively, satisfy the

full browser-level MACHINE OK conjunction. The difficult part is therefore not answer text alone; it is the simultaneous construction of an algorithm execution, its visual representation, a working browser program, and a stateful teaching interaction.

We diagnose this failure mode as *constraint entanglement*. A monolithic generator must satisfy answer, trace, scene, runtime, feedback, and noninterference obligations in one open-ended artifact:

$$C_{\text{answer}}, C_{\text{trace}}, C_{\text{scene}}, \\ C_{\text{runtime}}, C_{\text{feedback}}, C_{\text{noninterference}}.$$

These obligations are implemented by the same HTML, JavaScript, data literals, and event handlers. A repair intended to make a button work can overwrite a correct answer; a visual rewrite can desynchronize a pointer; and adding feedback can mutate the execution frame. More generation budget does not by itself remove such cross-obligation interference.

ALGO-TUTORGEN addresses this problem with *contract-guided semantic factorization*. An LLM first produces an executable solution specification. Sandboxed materialization records a typed SEMANTICTRACE through a restricted tracing interface. Validators check the result, trace structure, process continuity, and scene bindings. A deterministic compiler maps the verified trace to a SCENEGRAPH, and a fixed WEB RUNTIME renders it. A separately constrained pedagogical overlay may change explanations, hints, prediction feedback, and learning-log state, but not the canonical algorithmic projection. A final browser audit checks the released page. The chain converts an opaque generation problem into explicit refinement obligations, where failures can be rejected at the boundary that exposes them.

This factorization supports two forms of reasoning. First, if the trace-to-scene and scene-to-runtime projections agree at every reachable frame, semantic preservation follows by transitivity. Second, if pure pedagogical actions leave algorithm state unchanged and navigation selects only verified frames, arbitrary finite interaction sequences preserve a runtime invariant. These are conditional statements about the abstract contracts, not a machine-assisted proof that every source trace is correct. We connect each premise to executable checks and state the remaining assumptions. We also analyze an ideal stage-local recovery policy, but the current implementation does not satisfy its full checkpointability

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

premise.

The evidence follows the same separation. On a paired 200-task, 23-family benchmark, ALGO_{TUTOR}GEN improves MACHINE OK by 50.0 percentage points over Direct HTML (95% paired-bootstrap confidence interval (CI) [43.0, 57.0], exact McNemar $p = 4.06 \times 10^{-29}$). All 55,108 evaluated representation frames agree under the canonical projection; all 2,198 tested semantic mutations are rejected while all 392 tested semantics-preserving transformations are accepted; and no noninterference counterexample is found in 1,561,298 browser actions. Cross-model and held-out experiments retain the reliability gap. By contrast, the recovery experiment is negative: retaining a solution specification does not produce a significant success advantage over restarting, because materialization and teaching are recomputed.

Our contributions are: (1) formulating interactive tutor generation as constraint-entangled artifact synthesis; (2) a contract-guided factorization separating generated semantics, deterministic compilation, browser execution, and teaching state; (3) conditional preservation and noninterference results plus the assumptions for ideal local recovery; and (4) evaluation of executable reliability, contract survival, semantic discrimination, noninterference, model variation, held-out tasks, ablations, scalability, and the negative recovery result.

Problem Formulation

This section defines the synthesized artifact and the obligations that distinguish an executable tutor from a page containing a correct answer.

Inputs, Artifacts, and Contracts

An input is $x = (q, i, e, c)$: algorithmic problem q , concrete instance i , optional oracle e , and optional reference code or strategy c . The internal synthesis bundle is $A = (S, T, G, P, R)$, comprising executable specification S , SEMANTICTRACE T , SCENEGRAPH sequence G , pedagogical overlay P , and browser runtime R . Deterministic export turns A into the released webpage W .

We require the conjunction

$$\Phi(A, W, x) = \Phi_{\text{result}} \wedge \Phi_{\text{trace}} \wedge \Phi_{\text{scene}} \wedge \Phi_{\text{runtime}} \wedge \Phi_{\text{feedback}} \wedge \Phi_{\text{noninterference}}. \quad (1)$$

Φ_{result} compares the computed result with e when an oracle is available. Φ_{trace} checks the typed event sequence, final-state consistency, and generic causal or state-continuity conditions. Φ_{scene} checks that visual objects, marks, pointers, edges, and frames bind to trace entities. Φ_{runtime} covers loading and reachability. Φ_{feedback} requires distinguishable correct and incorrect responses plus hint, answer, and log behavior. $\Phi_{\text{noninterference}}$ requires the teaching layer to be read-only with respect to algorithm facts, except that navigation may select another verified frame.

Monolithic and Factored Search

Direct HTML asks a model to construct a point in an open space Ω_{HTML} whose textual answer, embedded data, rendering logic, event handlers, and teaching state jointly satisfy Equation (1). A defect is often observable only after a

browser executes the whole artifact, and a whole-file rewrite can invalidate an obligation that was previously satisfied.

ALGO_{TUTOR}GEN instead constructs a refinement chain

$$\begin{aligned} S &\xrightarrow{C_S} T \xrightarrow{C_T} G \xrightarrow{C_G} R, \\ G &\xrightarrow{\text{teaching}} P \xrightarrow{C_P} \bar{P}, \\ (R, \bar{P}) &\xrightarrow{C_B} W, \end{aligned}$$

where each C is an executable contract at a representation boundary. This notation does not claim that writing the output space as a Cartesian product automatically reduces search complexity. The benefit comes from four operational choices: contracts reject invalid candidates before release; verified semantic data are reused rather than regenerated; deterministic stages remove open-ended choices from downstream code; and failures are attributed to a narrower boundary. Modularization without these properties would merely rearrange the same entanglement.

System Design

This section explains how ALGO_{TUTOR}GEN assigns one semantic responsibility to each intermediate representation and where the implementation remains less modular than the idealized architecture.

The open-ended stage emits an executable solution specification that solves the instance and records events through `TraceSession`. A sandbox restricts imports, time, serialization, and communication. Materialization produces typed snapshots containing stable identifiers, values, marks, pointers, graph edges, containers, steps, and final answers, but no CSS or event handlers. Result and trace validators check oracle agreement, schema, and internal consistency. The process validator checks generic DSL causality and continuity; it is not a proved operational semantics for all 23 families, so stronger source-trace claims still require family or human audit.

A deterministic compiler binds trace objects to scene cells, nodes, edges, pointers, highlights, queues, stacks, and annotations. The online scene validator checks schema, nonempty frames, object-ID uniqueness, visible objects, and the integrity of mark, parent, source, and target references. Canonical Trace–SceneGraph–Runtime projection agreement is measured separately by the offline preservation audit; it is not part of this structural scene gate. The shared WEB RUNTIME supplies layout, navigation, variants, and rendering in a self-contained page. Teaching enrichment receives sanitized verified facts and may change objectives, explanations, predictions, hints, answer reveals, and logs, but fields redefining algorithm state are removed or rejected. The browser release audit checks the nine MACHINE OK conditions. On failure, LOCAL RESUME retains the specification but reruns materialization and teaching; it is not five-stage checkpoint recovery (Section).

Theoretical Analysis

This section states what follows from the representation contracts. The results are paper-and-pencil implications of ex-

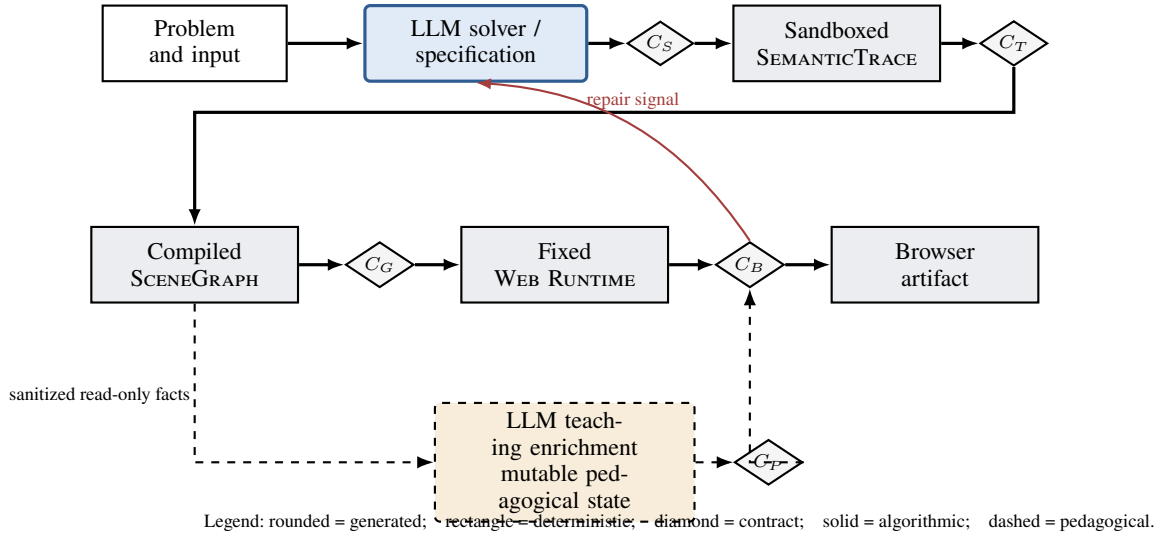


Figure 1: ALGO TutorGen architecture. Rounded blue boxes are open-ended LLM stages; rectangular gray boxes are deterministic stages; diamonds are executable contracts. Solid flow carries canonical algorithmic state. Dashed flow carries mutable pedagogical state derived from a sanitized, read-only view. The repair loop shown is the implemented specification-level boundary, not a fully checkpointed retry at every stage.

licit premises. They are not claims of machine-checked verification for arbitrary generated traces.

Canonical Algorithmic Projection

Definition 1 (Canonical algorithmic projection). *Let $\mathcal{A}$ be a representation-independent algorithm-state space containing stable object identifiers, values, marks, pointers, graph edges, stack or queue contents, the current step, and the final answer. For trace frame T_t , scene frame G_t , and reachable runtime state R_t , define*

$$\alpha_T : T_t \rightarrow \mathcal{A}, \quad \alpha_G : G_t \rightarrow \mathcal{A}, \quad \alpha_R : R_t \rightarrow \mathcal{A}.$$

Layout coordinates, CSS, explanatory prose, hint visibility, prediction text, and learning-log contents are excluded from $\mathcal{A}$.

The implementation serializes these projections in a canonical order before comparison and hashing. An equality therefore means equality of normalized algorithm facts, not pixel equality.

Compositional Semantic Preservation

Theorem 1 (Compositional Semantic Preservation). *For input x , let $S(x)$ be a verified solver result, let $T = (T_0, \dots, T_n)$ be a semantic trace, let $G = (G_0, \dots, G_n)$ be its compiled scene sequence, and let $R(G)$ be the rendered runtime execution. Suppose*

$$\begin{aligned} \text{Final}(T) &= S(x), \\ \alpha_T(T_t) &= \alpha_G(G_t) && \text{for every } t, \\ \alpha_G(G_t) &= \alpha_R(R_t) && \text{for reachable } t. \end{aligned}$$

Then $\alpha_T(T_t) = \alpha_R(R_t)$ for every reachable t , and the rendered final answer equals $S(x)$. If additionally $S(x) = \text{Oracle}(x)$, the rendered final answer equals the oracle.

Proof. Transitivity gives $\alpha_T(T_t) = \alpha_R(R_t)$ at each reachable t . At the final frame its answer component is $\text{Final}(T) = S(x)$; substituting oracle equality proves the last claim. $\square$

Theorem 1 is a layer-to-layer preservation theorem. It does not establish that an arbitrary T is the intended stepwise algorithm. The result and trace checks support $\text{Final}(T) = S(x)$, and the projection audit checks the two representation equalities on finite artifacts. Correctness of every source event relative to an independent algorithm semantics remains an assumed or separately audited premise.

Pedagogical Noninterference

We adapt the noninterference idea of separating protected and observable state (Goguen and Meseguer 1982) to a user-interface setting. Let runtime state be $\sigma = (a, p)$, where $a \in \mathcal{A}$ is algorithmic state and p is pedagogical state. Pure pedagogical actions are `submit_correct`, `submit_wrong`, `hint`, `show_answer`, and `clear_learning_log`. Navigation uses `next`, `previous`, and `timeline`; it also includes `reset` and `select_variant`.

Theorem 2 (Pedagogical Noninterference). *Suppose every pure pedagogical action u satisfies*

$$\delta_u(a, p) = (a, p')$$

for some p' . Then, for every finite sequence of pure pedagogical actions $u_1, \dots, u_m$,

$$\pi_A(\delta_{u_m} \circ \dots \circ \delta_{u_1}(a, p)) = a.$$

Moreover, if each navigation action selects an algorithm state from the verified trajectory

$$\mathcal{V}(T) = \{\alpha_T(T_0), \dots, \alpha_T(T_n)\},$$

then after any finite mixed sequence of pure pedagogical and navigation actions, the algorithm state belongs to $\mathcal{V}(T)$.

Proof. Induct on sequence length. A pure action preserves a ; a navigation action replaces the current member of $\mathcal{V}(T)$ with another member. Each induction step therefore preserves the corresponding invariant. $\square$

The theorem describes the intended abstract runtime. The concrete browser stress test checks for violations of its premises through state, step, and artifact hashes; passing finite tests is not a proof for all future code or action sequences.

Nested Contract Survival

Proposition 1 (Nested contract survival). *For sequential contract events $C_1, \dots, C_m$ with nonzero conditioning probabilities,*

$$\Pr\left(\bigcap_{i=1}^m C_i\right) = \prod_{i=1}^m \Pr\left(C_i \mid \bigcap_{j=1}^{i-1} C_j\right).$$

This is the probability chain rule, not a new result. Its diagnostic value is that cumulative survival localizes the first lossy boundary. Figure 2 shows large monolithic losses at runtime, interaction, and feedback, while factored survival stays near one after semantic validation.

Conditional Stage-Local Recovery

Theorem 3 (Ideal stage-local recovery). *Consider k sequential stages with cost $c_i > 0$ and independent, stationary per-attempt success probability $p_i \in (0, 1]$. Suppose a validator identifies the failing stage exactly; every successful stage is checkpointable; a failed stage can be retried independently; retry does not invalidate prior stages; and verified downstream work is not recomputed. Then*

$$\mathbb{E}[C_{\text{local}}] = \sum_{i=1}^k \frac{c_i}{p_i}, \quad \mathbb{E}[C_{\text{global}}] = \sum_{i=1}^k \frac{c_i}{\prod_{j=i}^k p_j},$$

and $\mathbb{E}[C_{\text{local}}] \leq \mathbb{E}[C_{\text{global}}]$.

Proof. Geometric retry yields local multiplicity $1/p_i$ and global multiplicity $1/\prod_{j=i}^k p_j$. Summation gives both costs; $\prod_{j=i}^k p_j \leq p_i$ proves the inequality. $\square$

This theorem is a conditional characterization, not a performance guarantee for the current system. Repairing a solution specification triggers materialization and teaching again, violating full checkpointability and the no-recomputation premise. The negative experiment in Section 3 is therefore a test of the implementation boundary, not a contradiction of Theorem 3.

Experimental Setup

This section defines the evaluation protocol and the meaning of each reported metric.

Tasks and Generation Conditions

The benchmark contains 200 tasks from 23 algorithm families and 646 concrete samples. The paired main comparison uses sample index 0 for each task, avoiding overweighting tasks that have more stored samples. The original main ALGO-TUTORGEN condition uses DeepSeek-V4-Pro. Cross-model final-quality experiments use DeepSeek-V4-Flash, GLM-5.2, and Kimi-K2.5. They begin with two candidates and two repair rounds; failed tasks may receive a recorded final-quality retry of at most three candidates and three rounds. The reported cross-model table uses the resulting frozen pages and is therefore not a uniform-call or token-matched comparison. A frozen held-out set contains 40 new cases from 15 families.

The Direct HTML prompt receives the problem, concrete input, and expected answer, and explicitly requests a complete self-contained teaching page with timeline, correct and incorrect feedback, hint, answer reveal, and learning log. Thus it is not denied the oracle or teaching requirements. Both approaches may use their specified repair policy. The main table uses selected-final artifacts after allowed retries: ALGO-TUTORGEN initially generated 195/200 primary pages and filled five missing artifacts with recorded targeted retries. Direct generated all selected pages within its allowed calls. This boundary prevents interpreting 200 available pages as first-pass generation success.

WebGen-Agent and strict Direct-plus-HTMLCure are external code paths adapted to the same browser audit. Their numbers measure performance under this task and audit protocol, not a reproduction of the authors’ original benchmark claims. External network resources are blocked during final evaluation so a page must be self-contained.

Metrics and Statistical Tests

MACHINE OK conjoins page load, visible-answer match, reachable interaction, correct feedback, wrong feedback, hint, answer reveal, learning log, and a lightweight answer-protection check. The ninth check compares the protected visible-answer summary before and after the audited teaching actions; the stronger algorithm-state, step, and artifact-hash comparison is reported separately in the noninterference stress test. We report exact counts; paired binary differences use exact two-sided McNemar tests, percentage-point differences use paired bootstrap CIs, and test families use Holm correction. Secondary paired scores use Wilcoxon tests where applicable. None measures student learning.

Theory-Aligned Audits

The preservation audit compares normalized α_T , α_G , and α_R for every frame in 200 main artifacts, a completed 40-artifact held-out set, and 54 long-trace artifacts. The mutation audit injects semantic violations and semantics-preserving changes. The determinism audit recompiles and rerenders 20 artifacts ten times each. The noninterference audit runs 100 random mixed action sequences on each of 240 pages and compares algorithm-state, step, and artifact hashes after every action.

The held-out generation experiment originally produced 39/40 valid ALGO-TUTORGEN artifacts. A targeted retry later

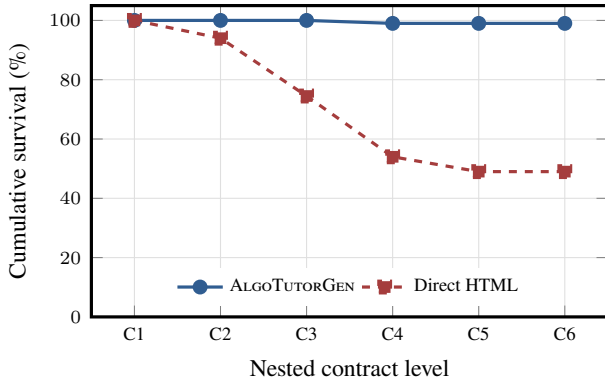


Figure 2: Nested contract survival on the main benchmark. C1: answer; C2: load; C3: reachable interaction; C4: bidirectional feedback; C5: hint, answer reveal, and log; C6: protected-answer stability. Values are cumulative, not independent per-contract accuracies; full-state noninterference is audited separately.

supplied the fortieth artifact only for preservation and noninterference coverage. We keep 39/40 as the generation result and label the 40/40 collection as a completed audit set.

Results

We organize the results by the obligation being tested: complete executable reliability, survival through nested contracts, representation-level semantics, interaction isolation, and robustness to model or task variation.

RQ1: Does Factorization Improve Executable Reliability?

Table 1 shows that answer visibility alone saturates for both principal methods, but full behavior does not. ALGO-TUTORGEN reaches 198/200 (99.0%) MACHINE OK versus 98/200 (49.0%) for Direct HTML, a paired difference of 50.0 percentage points (95% CI [43.0, 57.0]). There are 101 pairs where only ALGO-TUTORGEN passes, one where only Direct passes, and 98 concordant pairs; exact McNemar $p = 4.06 \times 10^{-29}$. The difference is concentrated in executable interaction and feedback rather than the displayed result.

The external paths obtain 45/200 and 40/200 under the same strict conjunction. Their failure profiles differ: WebGen-Agent usually loads but often misses answer or feedback requirements, whereas the strict HTMLCure path loses many pages before full interaction evaluation. These results support a protocol-specific comparison of released artifacts, not a universal ranking of website-generation systems.

RQ2: Where Do Monolithic Artifacts Fail?

Figure 2 applies Proposition 1. ALGO-TUTORGEN remains at 100% through interaction and loses two pages at feedback, after which the surviving set remains stable. Direct HTML drops from 100% answer match to 94.0% load, 74.5% interaction, 54.0% bidirectional feedback, and 49.0% complete

teaching support. The same qualitative pattern appears under Flash, GLM, and Kimi: the factored pipeline retains high downstream survival, while Direct loses additional artifacts at interaction and feedback boundaries. This evidence localizes constraint entanglement; it does not imply independence among contracts.

RQ3: Are Semantics Preserved and Contracts Discriminative?

The main, completed held-out, and long-trace sets contribute 9,421, 4,568, and 41,119 agreeing frames, respectively. This supports Theorem 1’s representation premises but does not prove that every source event is the intended algorithm step. The mutation suite rejects all 2,198 defined semantic violations and accepts all 392 preserving transformations (195 teaching rewrites, 195 visual-metadata changes, and two equivalent reorderings). We call this *contract discrimination* on the defined suite, not universal soundness and completeness. Twenty artifacts also produce one projection and render hash each across ten reruns.

RQ4: Does the Concrete Runtime Respect Teaching Isolation?

The suite executes 24,000 random sequences and 1,561,298 actions on 240 pages: 435,859 pure pedagogical actions preserve algorithm state, step, and artifact hash, while 1,125,439 navigation actions land on verified frames. All overlays pass sanitization; illegal algorithm fields are removed or rejected. No counterexample is found, but finite randomized execution does not establish noninterference for all future code.

RQ5: Does the Reliability Gap Persist?

Table 3 shows positive paired differences for each model and the held-out set; varying absolute success reflects strict gate rejection, model variation, and the recorded final-quality retry policy, not model invariance or equal compute. On held-out tasks, Direct generates 40/40 self-contained pages but only 18 pass, whereas ALGO-TUTORGEN produces 39 valid pages and all pass. A later targeted completion supplies audit coverage only and does not change the original 39/40 result.

Ablation Summary

Two 50-task ablations isolate the representation boundaries. Replacing executable tracing with Direct-to-SceneGraph reduces MACHINE OK from 49/50 to 1/50; replacing deterministic compilation with free HTML from a verified trace reduces it from 49/50 to 0/50 despite 50/50 visible answers. Neither a fixed runtime nor a correct trace alone is sufficient.

Negative Result: Recovery Is Not Yet Decoupled

This section tests whether the implementation meets the checkpoint premise of Theorem 3. The result is negative and materially changes the system claim.

For Flash, GLOBAL RESTART is numerically better in both success (42/50 versus 38/50) and tokens per success (62,256 versus 71,369), although its mean successful completion time

Table 1: Main 200-task browser reliability. Every entry is a count out of 200. MACHINE OK is the conjunction of nine checks, including the five displayed component columns and hint, answer reveal, learning log, and protected-answer stability.

Method	Load	Visible answer	Interaction reachable	Correct feedback	Wrong feedback	MACHINE OK
ALGO TUTOR GEN	200	200	200	199	198	198
Direct HTML	188	200	149	120	125	98
WebGen-Agent	194	169	154	74	89	45
Direct + HTMLCure (strict)	75	75	62	52	51	40

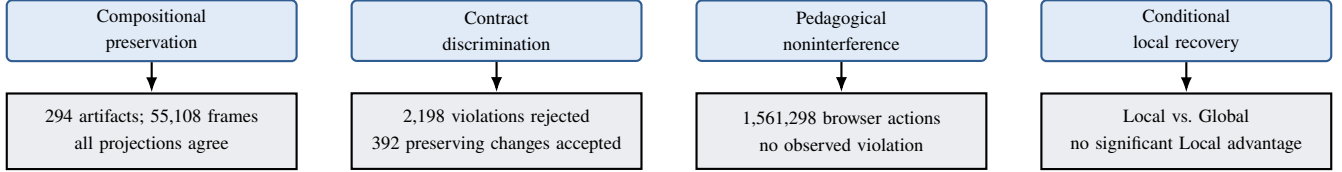


Figure 3: Theory-to-evidence map. Projection equality does not prove source-trace correctness; mutation and browser results cover finite suites; recovery is negative because checkpointability is absent.

is higher. For GLM, LOCAL RESUME uses slightly fewer tokens and less time per successful page, but succeeds on one fewer task. Neither paired success comparison is significant. We therefore find no evidence that the implemented Local policy improves success, and no consistent cost advantage.

Instrumentation explains the mismatch between the ideal theorem and the measured system. Local repair reduces solution-specification generation and repair work, but the repaired specification is materialized again and teaching enrichment is invoked again. Those calls can dominate or erase the saved work. In addition, a locally repaired candidate may preserve a specification that is less amenable to downstream teaching than a fresh global draw. The implementation provides specification-level repair, not independently checkpointed solver, trace, scene, runtime, and teaching stages.

The result does not contradict Theorem 3; it shows that its strongest premises are absent. More importantly, it distinguishes three concepts that are easy to conflate. ALGOTUTORGEN substantially realizes *semantic decoupling* and *verification decoupling*. It does not yet realize full *computational recovery decoupling*. Semantic factorization is therefore necessary infrastructure for local recovery, but it does not automatically supply computational checkpointing.

Related Work

Generated webpages and educational visualization. Design2Code and WebGen-Bench evaluate front-end reconstruction or functional website generation (Si et al. 2025; Lu et al. 2025b); WebGen-Agent and HTMLCure add iterative browser feedback and repair (Lu et al. 2025a; HTMLCure Contributors 2026), while EduVisAgent studies pedagogical-visualization planning (Ji et al. 2025). Our task is narrower, but each page is compiled from an executable trace and audited for cross-representation state and teaching isolation; external-baseline results therefore apply only to our protocol.

Algorithm-visualization work distinguishes passive view-

ing from active engagement and cautions that visualization alone does not ensure learning (Naps et al. 2002; Hundhausen, Douglas, and Stasko 2002). TRAKLA2, Python Tutor, JSAV, OpenDSA, VisuAlgo, JHAVE, and Algorithm Visualizer provide assessed exercises, execution views, libraries, or curated content (Malmi et al. 2004; Guo 2013; Karavirta and Shaffer 2013; Shaffer et al. 2011; Halim 2015; Naps 2005; Algorithm Visualizer Contributors 2026); surveys characterize this broader design space (Sorva, Karavirta, and Malmi 2013). These systems generally use human-authored algorithms or templates, whereas ALGOTUTORGEN synthesizes benchmark-task artifacts. Multimedia learning, LORI, and visualization-validation principles inform interface analysis (Mayer 2009; Leacock and Nesbit 2007; Munzner 2009), but do not justify a learning-gain claim here.

Synthesis, contracts, and testing. Program synthesis and systems such as Sketch and SYNQUID use explicit specifications to constrain search (Gulwani, Polozov, and Singh 2017; Solar-Lezama et al. 2006; Polikarpova, Kuraj, and Solar-Lezama 2016). Interface automata and modular temporal reasoning expose component assumptions (de Alfaro and Henzinger 2001; Pnueli 1985); CompCert proves compiler preservation mechanically (Leroy 2009). In contrast, our preservation theorem is conditional and its implementation evidence is finite. We borrow state separation from noninterference (Goguen and Meseguer 1982) and counterexample search from QuickCheck and metamorphic testing (Claessen and Hughes 2000; Segura et al. 2016), without claiming an information-flow proof.

Discussion and Limitations

Factorization works because intermediate representations expose comparable semantic objects, contracts reject defects early, deterministic compilation reuses verified facts, and teaching receives a sanitized read-only view. The ablations show that neither a fixed runtime without executable semantics nor a verified trace followed by free HTML is sufficient;

Table 2: Theory-aligned executable evidence. The rightmost column states the strongest warranted interpretation.

Property and scope	Observed outcome	Evidence boundary
Projection, 294 artifacts	55,108/55,108 frames agree	Representation preservation on evaluated frames; source-trace correctness is a separate premise.
Determinism, 20 artifacts $\times$ 10	One projection and render hash per artifact	Stable in the tested environment, not a cross-platform bitwise claim.
Semantic violations	2,198/2,198 rejected	Complete for defined mutation operators, not universal validator soundness.
Preserving transformations	392/392 accepted	Complete for 195 teaching rewrites, 195 visual-metadata changes, and two equivalent reorderings.
Noninterference, 240 pages	1,561,298 actions; zero violations	No counterexample in 24,000 sequences, not proof of every execution.

Table 3: Cross-model final-quality and frozen held-out MACHINE OK results. Cross-model rows use 200 paired tasks; ALGO-TUTORGEN starts from a two-candidate, two-repair budget and applies recorded final-quality retries to failed tasks, so calls and tokens are not matched to Direct. The held-out row uses 40 new cases.

Model or set	ALGO-TUTORGEN	Direct	Difference	95% paired-bootstrap CI	Exact McNemar p
DeepSeek-V4-Flash	200/200 (100.0%)	118/200 (59.0%)	+41.0 pp	[34.5, 48.0]	4.14×10^{-25}
GLM-5.2	196/200 (98.0%)	35/200 (17.5%)	+80.5 pp	[75.0, 86.0]	2.81×10^{-47}
Kimi-K2.5	194/200 (97.0%)	87/200 (43.5%)	+53.5 pp	[46.0, 60.5]	4.79×10^{-30}
Held-out, 15 families	39/40 (97.5%)	18/40 (45.0%)	+52.5 pp	[37.5, 67.5]	9.54×10^{-7}

reliability comes from the complete refinement chain.

Mathematically, Theorems 1 and 2 establish conditional refinement and state invariants, while Theorem 3 assumes ideal checkpointing. Experimentally, all evaluated projections agree, the defined mutation suite discriminates violations from preserving changes, and no noninterference counterexample appears in 1,561,298 actions. These results do not prove arbitrary source traces correct, establish universal validator guarantees, or ensure future browser executions are safe. The recovery result adds a systems lesson: semantic factorization does not automatically imply computational checkpointing. Future recovery should persist versioned stage artifacts and invalidate only descendants of changed semantic objects.

MACHINE OK measures executable completeness, not teaching quality or learning, and may favor the fixed runtime. Two-annotator calibration of 120 pages and an independent 40-task trace audit remain unlabeled. Mutation operators and action sequences are finite; the main study uses one sample per task, the 646-sample replay is unbalanced and contains failures, and held-out evaluation has 40 cases. Compute also differs: selected-final ALGO-TUTORGEN averages 5.33 calls and 76.8k tokens per task versus 1.11 calls and 21.9k for Direct, while the fixed runtime embodies additional engineering.

Remote models can drift and external baselines are task-adapted. Full-frame export also scales poorly: large traces average 1,636.7 frames, 160.42 MB, 8.354 s load, 101.3 ms step latency, and 185.6 MB heap; two extremes reach 581 MB and 1.08 GB and exceed 60 s. Delta encoding and virtualization are needed. Stage2 visual ratings use one vision-language evaluator, and expert and 24-student studies remain protocols. No usability, retention, transfer, or learning-gain result is claimed.

Conclusion

Interactive algorithm tutors are constraint-entangled artifacts: algorithm facts, execution traces, visual states, browser behavior, feedback, and teaching-state isolation must all hold together. ALGO-TUTORGEN replaces monolithic HTML generation with contract-guided semantic factorization through an executable specification, semantic trace, validated scene, deterministic runtime, and isolated teaching overlay. Under the evaluated protocol, this architecture raises complete browser reliability from 98/200 to 198/200 while preserving canonical projections for all 55,108 evaluated frames and revealing no pedagogical noninterference counterexample in 1.56 million actions. These are strong finite executable results linked to conditional compositional theorems, not formal verification of arbitrary traces or evidence of student learning. The negative recovery experiment sharpens the conclusion: semantic and verification decoupling do not automatically provide computational checkpointing. Building truly stage-local, dependency-aware recovery is the next systems boundary.

Table 4: LOCAL RESUME versus GLOBAL RESTART on paired 50-task sets. Tokens and time are normalized by successful pages. The McNemar value compares success within each model; nonsignificance is not evidence of equivalence.

Model	Strategy	Success	Tokens/success	Mean time to valid	Exact McNemar p
DeepSeek-V4-Flash	LOCAL RESUME	38/50 (76.0%)	71,369	172.9 s	0.4545
	GLOBAL RESTART	42/50 (84.0%)	62,256	194.2 s	
GLM-5.2	LOCAL RESUME	42/50 (84.0%)	92,385	533.8 s	1.0000
	GLOBAL RESTART	43/50 (86.0%)	96,186	558.2 s	

References

- Algorithm Visualizer Contributors. 2026. Algorithm Visualizer. Official source repository, commit 18de2edf6b629e843ee4bcedfa2208d22b253f0f.
- Claessen, K.; and Hughes, J. 2000. QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming*, 268–279. ACM.
- de Alfaro, L.; and Henzinger, T. A. 2001. Interface Automata. In *Proceedings of the 8th European Software Engineering Conference Held Jointly with the 9th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 109–120. ACM.
- Goguen, J. A.; and Meseguer, J. 1982. Security Policies and Security Models. In *1982 IEEE Symposium on Security and Privacy*, 11–20. IEEE.
- Gulwani, S.; Polozov, O.; and Singh, R. 2017. Program Synthesis. *Foundations and Trends in Programming Languages*, 4(1–2): 1–119.
- Guo, P. J. 2013. Online Python Tutor: Embeddable Web-Based Program Visualization for CS Education. In *Proceedings of the 44th ACM Technical Symposium on Computer Science Education*, 579–584. ACM.
- Halim, S. 2015. VisuAlgo: Visualising Data Structures and Algorithms Through Animation. *Olympiads in Informatics*, 9: 243–245.
- HTMLCure Contributors. 2026. HTMLCure: Experience-Driven Evaluation and State-Aware Repair for LLM-Generated Interactive HTML. Official source repository, commit 18d68e8f1e5c2bcef7f3c00bcab3147e2a99d4db.
- Hundhausen, C. D.; Douglas, S. A.; and Stasko, J. T. 2002. A Meta-Study of Algorithm Visualization Effectiveness. *Journal of Visual Languages and Computing*, 13(3): 259–290.
- Ji, H.; Qiu, S.; Xin, S.; Han, S.; Chen, Z.; Zhang, D.; Wang, H.; and Yao, H. 2025. From EduVisBench to EduVisAgent: A Benchmark and Multi-Agent Framework for Reasoning-Driven Pedagogical Visualization. arXiv:2505.16832.
- Karavirta, V.; and Shaffer, C. A. 2013. JSAV: The JavaScript Algorithm Visualization Library. In *Proceedings of the 18th ACM Conference on Innovation and Technology in Computer Science Education*, 159–164. ACM.
- Leacock, T. L.; and Nesbit, J. C. 2007. A Framework for Evaluating the Quality of Multimedia Learning Resources. *Educational Technology and Society*, 10(2): 44–59.
- Leroy, X. 2009. Formal Verification of a Realistic Compiler. *Communications of the ACM*, 52(7): 107–115.
- Lu, Z.; Ren, H.; Yang, Y.; Wang, K.; Zong, Z.; Pan, J.; Zhan, M.; and Li, H. 2025a. WebGen-Agent: Enhancing Interactive Website Generation with Multi-Level Feedback and Step-Level Reinforcement Learning. arXiv:2509.22644.
- Lu, Z.; Yang, Y.; Ren, H.; Hou, H.; Xiao, H.; Wang, K.; Shi, W.; Zhou, A.; Zhan, M.; and Li, H. 2025b. WebGen-Bench: Evaluating LLMs on Generating Interactive and Functional Websites from Scratch. arXiv:2505.03733.
- Malmi, L.; Karavirta, V.; Korhonen, A.; Nikander, J.; Sepälä, O.; and Silvasti, P. 2004. Visual Algorithm Simulation Exercise System with Automatic Assessment: TRAKLA2. *Informatics in Education*, 3(2): 267–288.
- Mayer, R. E. 2009. *Multimedia Learning*. Cambridge University Press, 2 edition. ISBN 9780521735353.
- Munzner, T. 2009. A Nested Model for Visualization Design and Validation. *IEEE Transactions on Visualization and Computer Graphics*, 15(6): 921–928.
- Naps, T. L. 2005. JHAVÉ: Supporting Algorithm Visualization. *IEEE Computer Graphics and Applications*, 25(5): 49–55.
- Naps, T. L.; Röbling, G.; Almstrum, V.; Dann, W.; Fleischer, R.; Hundhausen, C.; Korhonen, A.; Malmi, L.; McNally, M.; Rodger, S.; and Velázquez-Iturbide, J. Á. 2002. Exploring the Role of Visualization and Engagement in Computer Science Education. *ACM SIGCSE Bulletin*, 35(2): 131–152.
- Pnueli, A. 1985. In Transition from Global to Modular Temporal Reasoning about Programs. In *Logics and Models of Concurrent Systems*, 123–144. Springer.
- Polikarpova, N.; Kuraj, I.; and Solar-Lezama, A. 2016. Program Synthesis from Polymorphic Refinement Types. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 522–538. ACM.
- Segura, S.; Fraser, G.; Sanchez, A. B.; and Ruiz-Cortes, A. 2016. A Survey on Metamorphic Testing. *IEEE Transactions on Software Engineering*, 42(9): 805–824.
- Shaffer, C. A.; Karavirta, V.; Korhonen, A.; and Naps, T. L. 2011. OpenDSA: Beginning a Community Active-eBook Project. In *Proceedings of the 11th Koli Calling International Conference on Computing Education Research*, 112–117. ACM.
- Si, C.; Zhang, Y.; Li, R.; Yang, Z.; Liu, R.; and Yang, D. 2025. Design2Code: Benchmarking Multimodal Code Generation for Automated Front-End Engineering. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human*

Language Technologies, 3956–3974. Association for Computational Linguistics.

Solar-Lezama, A.; Tancau, L.; Bodik, R.; Seshia, S.; and Saraswat, V. 2006. Combinatorial Sketching for Finite Programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*, 404–415. ACM.

Sorva, J.; Karavirta, V.; and Malmi, L. 2013. A Review of Generic Program Visualization Systems for Introductory Programming Education. *ACM Transactions on Computing Education*, 13(4): 1–64.